

Room XI Connect - Security & Production Hardening Guide

Version: 1.0
Date: January 6, 2026
Audience: DevOps, Security, Backend Engineers
Status: Critical - Implement Before Production

Table of Contents

- 1. [Security Assessment](#)
- 2. [Authentication & Authorization](#)
- 3. [Data Protection](#)
- 4. [API Security](#)
- 5. [Infrastructure Security](#)
- 6. [Compliance & Privacy](#)
- 7. [Monitoring & Incident Response](#)
- 8. [Hardening Checklist](#)

Security Assessment

Current Security Posture

Component	Status	Risk Level	Notes
Authentication	✅ Implemented	LOW	Session-based with JWT tokens
Authorization	✅ Implemented	LOW	Role-based access control
Encryption (Transit)	✅ HTTPS	LOW	TLS 1.3 required
Encryption (Rest)	⚠️ Partial	MEDIUM	Database encryption not verified
Input Validation	✅ Zod schemas	LOW	All inputs validated

CORS	✓ Configured	LOW	Proper origin whitelisting
Rate Limiting	✓ Implemented	LOW	Auth and write endpoints protected
CSRF Protection	✓ SameSite cookies	LOW	SameSite=Strict configured
SQL Injection	✓ ORM protection	LOW	Using Drizzle ORM
XSS Protection	✓ React escaping	LOW	Built-in React protection
Secrets Management	✗ Missing	CRITICAL	Secrets not in vault
Audit Logging	⚠ Partial	MEDIUM	Basic logging, not comprehensive
Error Handling	✓ Implemented	LOW	Generic error messages

Vulnerability Summary

Critical Issues:

- Missing production secrets (SESSION_SECRET, ENCRYPTION_SECRET)
- 2 HIGH severity dependency vulnerabilities
- Admin credentials in version control

Medium Issues:

- 10 MODERATE severity dependency vulnerabilities
- Database encryption not verified
- Limited audit logging

Low Issues:

- ESLint configuration scanning build artifacts
- TypeScript server configuration issues

Authentication & Authorization

Session Management

Current Implementation

TypeScript

```
// Session middleware (server/middleware/auth.ts)
const sessionMiddleware = session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: {
    secure: true,          // HTTPS only
    httpOnly: true,       // No JavaScript access
    sameSite: 'strict',   // CSRF protection
    maxAge: 7 * 24 * 60 * 60 * 1000 // 7 days
  }
});
```

Hardening Recommendations

1. Reduce session timeout

TypeScript

```
maxAge: 4 * 60 * 60 * 1000 // 4 hours instead of 7 days
```

2. Implement session rotation

TypeScript

```
// Regenerate session ID after login
req.session.regenerate((err) => {
  if (err) return next(err);
  // Set user data
});
```

3. Add session activity tracking

TypeScript

```
// Update last activity timestamp
req.session.lastActivity = Date.now();

// Invalidate if inactive for 30 minutes
```

```
if (Date.now() - req.session.lastActivity > 30 * 60 * 1000) {  
  req.session.destroy();  
}
```

4. Implement multi-factor authentication (MFA)

TypeScript

```
// For future: TOTP (Time-based One-Time Password)  
// Require MFA for sensitive operations (profile changes, etc.)
```

Password Security

Current Implementation

TypeScript

```
// Password hashing (bcrypt)  
const hashedPassword = await bcrypt.hash(password, 10);
```

Hardening Recommendations

1. Enforce strong password requirements

TypeScript

```
const passwordSchema = z.string()  
  .min(12, "Password must be at least 12 characters")  
  .regex(/[A-Z]/, "Must contain uppercase letter")  
  .regex(/[a-z]/, "Must contain lowercase letter")  
  .regex(/[0-9]/, "Must contain number")  
  .regex(/[!@#$$%^&*/, "Must contain special character");
```

2. Implement password history

- Prevent reuse of last 5 passwords
- Store hashed previous passwords

3. Add password expiration

- Require password change every 90 days
- Notify users 14 days before expiration

4. Implement account logout

TypeScript

```
// Lock account after 5 failed attempts
// Unlock after 30 minutes or admin action
```

Authorization & Access Control

Role-Based Access Control (RBAC)

TypeScript

```
enum UserRole {
  YOUTH = 'youth',
  GUARDIAN = 'guardian',
  FACILITATOR = 'facilitator',
  ADMIN = 'admin'
}

// Middleware for role checking
const requireRole = (...roles: UserRole[]) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  };
};
```

Permission Matrix

Action	Youth	Guardian	Facilitator	Admin
View own profile	✓	✓	✓	✓
Edit own profile	✓	✓	✓	✓
View mood history	✓	✓	-	✓
View other users	-	✓ (child)	✓ (enrolled)	✓
Create program	-	-	✓	✓
Edit program	-	-	✓ (own)	✓

Delete program	-	-	-	✓
View analytics	-	-	✓ (own)	✓
Manage users	-	-	-	✓
Access admin panel	-	-	-	✓

Data Protection

Encryption at Rest

Current Status

- ⚠ Database encryption status unknown
- ⚠ Backup encryption not verified

Recommendations

1. Enable Supabase encryption

SQL

```
-- Verify encryption is enabled
SELECT * FROM pg_settings WHERE name LIKE '%encrypt%';
```

2. Encrypt sensitive fields

TypeScript

```
// Use pgcrypto extension for field-level encryption
CREATE EXTENSION IF NOT EXISTS pgcrypto;

// Encrypt sensitive data
UPDATE users SET
  phone_encrypted = pgp_sym_encrypt(phone, 'encryption_key')
WHERE phone IS NOT NULL;
```

3. Backup encryption

- Enable Supabase backup encryption

- Store backup encryption keys in separate vault
- Test backup restoration regularly

Encryption in Transit

Current Implementation

-  HTTPS/TLS 1.3 enforced
-  Secure cookies (httpOnly, Secure, SameSite)

Recommendations

1. Enforce HTTPS everywhere

TypeScript

```
// Middleware to redirect HTTP to HTTPS
app.use((req, res, next) => {
  if (req.header('x-forwarded-proto') !== 'https' ) {
    return res.redirect(301, `https://${req.header('host' )}${req.url}`);
  }
  next();
});
```

2. Add HSTS header

TypeScript

```
app.use((req, res, next) => {
  res.setHeader('Strict-Transport-Security', 'max-age=31536000;
includeSubDomains; preload');
  next();
});
```

3. Add security headers

TypeScript

```
app.use((req, res, next) => {
  res.setHeader('X-Content-Type-Options', 'nosniff');
  res.setHeader('X-Frame-Options', 'DENY');
  res.setHeader('X-XSS-Protection', '1; mode=block');
  res.setHeader('Referrer-Policy', 'strict-origin-when-cross-origin');
```

```
    next();
  });
```

Data Minimization

Sensitive Data Handling

1. Limit data collection

- Only collect data necessary for functionality
- Obtain explicit consent for demographic data
- Allow users to skip optional fields

2. Mask sensitive data in logs

TypeScript

```
// Never log passwords, tokens, or PII
const sanitizeForLogging = (obj) => {
  const sensitiveFields = ['password', 'token', 'ssn', 'creditCard'];
  const sanitized = { ...obj };
  sensitiveFields.forEach(field => {
    if (sanitized[field]) {
      sanitized[field] = '***REDACTED***';
    }
  });
  return sanitized;
};
```

3. Data retention policies

TypeScript

```
// Delete data after retention period
// Moods: Keep for 2 years, then archive
// Check-ins: Keep for 1 year, then delete
// Logs: Keep for 90 days, then delete
```

API Security

Input Validation

Current Implementation

TypeScript

```
// Zod schema validation
const createMoodSchema = z.object({
  moodLevel: z.number().min(1).max(10),
  moodCategory: z.enum(['happy', 'sad', 'anxious', 'calm', 'angry']),
  notes: z.string().max(1000),
  triggers: z.array(z.string()).max(10)
});
```

Hardening Recommendations

1. Validate all inputs

TypeScript

```
// Middleware to validate request body
const validateRequest = (schema) => {
  return (req, res, next) => {
    try {
      req.body = schema.parse(req.body);
      next();
    } catch (error) {
      return res.status(400).json({ error: 'Invalid input' });
    }
  };
};
```

2. Sanitize user input

TypeScript

```
import DOMPurify from 'isomorphic-dompurify';

const sanitizeInput = (input) => {
  return DOMPurify.sanitize(input);
};
```

3. Prevent NoSQL injection

- Use parameterized queries (Drizzle ORM)
- Never concatenate user input into queries

Rate Limiting

Current Implementation

TypeScript

```
// Rate limiting middleware
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5, // 5 requests per window
  message: 'Too many login attempts'
});

const writeLimiter = rateLimit({
  windowMs: 60 * 1000, // 1 minute
  max: 30, // 30 requests per minute
  message: 'Too many requests'
});
```

Hardening Recommendations

1. Implement distributed rate limiting

TypeScript

```
// Use Redis for distributed rate limiting
const RedisStore = require('rate-limit-redis');
const redis = require('redis');

const limiter = rateLimit({
  store: new RedisStore({
    client: redis.createClient(),
    prefix: 'rl:'
  }),
  windowMs: 15 * 60 * 1000,
  max: 100
});
```

2. Add per-user rate limiting

TypeScript

```
// Different limits for different user roles
const getLimitByRole = (role) => {
  const limits = {
    youth: 100,
    guardian: 200,
    admin: 1000
  }
}
```

```
};  
return limits[role] || 50;  
};
```

CORS Configuration

Current Implementation

TypeScript

```
app.use(cors({  
  origin: process.env.ALLOWED_ORIGINS?.split(','),  
  credentials: true,  
  methods: ['GET', 'POST', 'PUT', 'DELETE'],  
  allowedHeaders: ['Content-Type', 'Authorization']  
}));
```

Hardening Recommendations

1. Whitelist specific origins

TypeScript

```
const allowedOrigins = [  
  'https://roomxi.live',  
  'https://app.roomxi.live',  
  'https://admin.roomxi.live'  
];  
  
app.use(cors({  
  origin: (origin, callback) => {  
    if (!origin || allowedOrigins.includes(origin)) {  
      callback(null, true);  
    } else {  
      callback(new Error('Not allowed by CORS'));  
    }  
  }  
}));
```

2. Restrict HTTP methods

TypeScript

```
// Only allow necessary methods  
app.use((req, res, next) => {
```

```
    if (!['GET', 'POST', 'PUT', 'DELETE', 'PATCH'].includes(req.method)) {
      return res.status(405).json({ error: 'Method not allowed' });
    }
    next();
  });
```

Error Handling

Current Implementation

TypeScript

```
// Global error handler
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ error: 'Internal server error' });
});
```

Hardening Recommendations

1. Don't expose sensitive error details

TypeScript

```
app.use((err, req, res, next) => {
  // Log full error internally
  logger.error({
    error: err.message,
    stack: err.stack,
    path: req.path,
    method: req.method,
    userId: req.user?.id
  });

  // Return generic error to client
  const statusCode = err.statusCode || 500;
  res.status(statusCode).json({
    error: statusCode === 500 ? 'Internal server error' : err.message
  });
});
```

2. Implement error tracking

TypeScript

```
import * as Sentry from "@sentry/node";

app.use(Sentry.Handlers.errorHandler());
```

Infrastructure Security

Vercel Security

Environment Variables

Bash

```
# Never commit secrets
# Use Vercel dashboard to set environment variables
# Or use Vercel CLI:
vercel env add SESSION_SECRET
vercel env add DATABASE_URL
vercel env add ENCRYPTION_SECRET
```

Deployment Security

Bash

```
# Enable branch protection
# Require reviews before merge
# Run security checks in CI/CD

# In GitHub Actions:
- name: Security Audit
  run: npm audit --audit-level=moderate

- name: SAST Scan
  run: npm run security:scan
```

Supabase Security

Row Level Security (RLS)

SQL

```

-- Enable RLS on all tables
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
ALTER TABLE moods ENABLE ROW LEVEL SECURITY;
ALTER TABLE programs ENABLE ROW LEVEL SECURITY;

-- Users can only see their own data
CREATE POLICY "Users can view own data"
ON users FOR SELECT
USING (auth.uid() = id);

-- Users can only update their own data
CREATE POLICY "Users can update own data"
ON users FOR UPDATE
USING (auth.uid() = id);

-- Moods are private to user
CREATE POLICY "Users can view own moods"
ON moods FOR SELECT
USING (auth.uid() = user_id);

-- Facilitators can view enrolled users' moods
CREATE POLICY "Facilitators can view enrolled users' moods"
ON moods FOR SELECT
USING (
  EXISTS (
    SELECT 1 FROM enrollments
    WHERE enrollments.user_id = moods.user_id
    AND enrollments.program_id IN (
      SELECT id FROM programs
      WHERE facilitator_id = auth.uid()
    )
  )
);

```

Database Backups

Bash

```

# Supabase automatically backs up daily
# Verify backup configuration in Supabase dashboard
# Test backup restoration monthly

# To restore from backup:
# 1. Go to Supabase dashboard

```

- # 2. Database → Backups
- # 3. Select backup and restore

Compliance & Privacy

GDPR Compliance

Data Subject Rights

1. Right to Access

TypeScript

```
// Endpoint to download user data
GET /api/users/:id/export
// Returns JSON with all user data
```

2. Right to Erasure (Right to be Forgotten)

TypeScript

```
// Endpoint to delete user account and data
DELETE /api/users/:id
// Anonymizes or deletes all personal data
```

3. Right to Data Portability

TypeScript

```
// Endpoint to export data in standard format
GET /api/users/:id/export?format=json
// Returns data in portable format
```

4. Right to Rectification

TypeScript

```
// Users can update their own data
PUT /api/users/:id
// Validate and update profile information
```

Consent Management

TypeScript

```
// Track user consent
const consentSchema = z.object({
  termsOfService: z.boolean(),
  privacyPolicy: z.boolean(),
  dataProcessing: z.boolean(),
  marketing: z.boolean().optional()
});

// Store consent records with timestamp
const consent = {
  userId: user.id,
  termsOfService: true,
  privacyPolicy: true,
  dataProcessing: true,
  marketing: false,
  consentedAt: new Date(),
  ipAddress: req.ip,
  userAgent: req.headers['user-agent']
};
```

PHIPA Compliance (Canada)

Privacy Impact Assessment

- ☐ Identify all personal health information (PHI)
- ☐ Document data collection, use, and storage
- ☐ Implement access controls
- ☐ Establish breach notification procedures
- ☐ Train staff on privacy requirements

Breach Notification

TypeScript

```
// Breach notification procedure
const notifyBreach = async (affectedUsers) => {
  // 1. Notify affected individuals within 30 days
  // 2. Document breach details
  // 3. Notify privacy commissioner if required
}
```



```
// 4. Implement remediation measures  
};
```

Monitoring & Incident Response

Error Tracking (Sentry)

Setup

TypeScript

```
// Initialize Sentry  
import * as Sentry from "@sentry/node";  
  
Sentry.init({  
  dsn: process.env.SENTRY_DSN,  
  environment: process.env.NODE_ENV,  
  tracesSampleRate: 1.0,  
  integrations: [  
    new Sentry.Integrations.Http({ tracing: true }),  
    new Sentry.Integrations.OnUncaughtException(),  
    new Sentry.Integrations.OnUnhandledRejection()  
  ]  
});  
  
// Capture exceptions  
try {  
  // Code  
} catch (error) {  
  Sentry.captureException(error);  
}
```

Logging

Structured Logging

TypeScript

```
import pino from 'pino';  
  
const logger = pino({  
  level: process.env.LOG_LEVEL || 'info',  
  transport: {
```

```
target: 'pino-pretty',
options: {
  colorize: true
}
});

// Log with context
logger.info({
  userId: user.id,
  action: 'mood_created',
  moodLevel: mood.level,
  timestamp: new Date()
});
```

Monitoring & Alerts

Key Metrics to Monitor

TypeScript

```
// Error rate
// API response time (p50, p95, p99)
// Database query time
// Memory usage
// CPU usage
// Disk usage
// Request rate
// Failed login attempts
// Unusual data access patterns
```

Alert Thresholds

Metric	Warning	Critical
Error Rate	>1%	>5%
API Response Time (p95)	>500ms	>2s
Database Query Time	>1s	>5s
Memory Usage	>80%	>95%
CPU Usage	>70%	>90%

Failed Logins (per user)	>3	>10
--------------------------	----	-----

Incident Response Plan

Incident Severity Levels

Level	Response Time	Escalation
P1 - Critical	15 minutes	CEO, CTO, Security
P2 - High	1 hour	CTO, Engineering Lead
P3 - Medium	4 hours	Engineering Team
P4 - Low	24 hours	Backlog

Incident Response Checklist

1. Detection & Alerting
 - ☐ Alert triggered
 - ☐ Incident created in tracking system
 - ☐ Team notified
2. Investigation
 - ☐ Gather logs and metrics
 - ☐ Identify root cause
 - ☐ Assess impact
3. Mitigation
 - ☐ Implement temporary fix
 - ☐ Verify fix effectiveness
 - ☐ Monitor for regression
4. Resolution
 - ☐ Implement permanent fix
 - ☐ Deploy to production

- ☐ Verify resolution

5. Post-Incident

- ☐ Document incident
 - ☐ Conduct post-mortem
 - ☐ Implement preventive measures
-

Hardening Checklist

Pre-Launch Security Checklist

- ☐ **Secrets Management**
 - ☐ All secrets removed from code
 - ☐ All secrets in environment variables
 - ☐ SESSION_SECRET configured
 - ☐ ENCRYPTION_SECRET configured
 - ☐ Database credentials secure
- ☐ **Dependency Security**
 - ☐ npm audit shows 0 vulnerabilities
 - ☐ All HIGH severity issues resolved
 - ☐ All MODERATE severity issues resolved
 - ☐ Dependencies updated to latest versions
- ☐ **Authentication**
 - ☐ Password hashing implemented (bcrypt)
 - ☐ Session management configured
 - ☐ CSRF protection enabled
 - ☐ Account lockout implemented
 - ☐ Password requirements enforced
- ☐ **Authorization**
 - ☐ RBAC implemented
 - ☐ Permission matrix defined

- ☐ Role-based access control tested
- ☐ Admin access restricted
- ☐ **Data Protection**
 - ☐ HTTPS/TLS enforced
 - ☐ Sensitive data encrypted
 - ☐ Data minimization implemented
 - ☐ Backup encryption enabled
 - ☐ Data retention policies defined
- ☐ **API Security**
 - ☐ Input validation implemented
 - ☐ Rate limiting configured
 - ☐ CORS properly configured
 - ☐ Error handling secure
 - ☐ SQL injection prevention verified
- ☐ **Infrastructure**
 - ☐ Vercel environment variables set
 - ☐ Supabase RLS policies enabled
 - ☐ Database backups tested
 - ☐ Monitoring configured
 - ☐ Alerts configured
- ☐ **Compliance**
 - ☐ GDPR compliance measures implemented
 - ☐ Privacy policy published
 - ☐ Terms of service published
 - ☐ Consent management implemented
 - ☐ Data breach notification plan ready
- ☐ **Monitoring**
 - ☐ Error tracking (Sentry) configured

- ☐ Logging configured
 - ☐ Health checks implemented
 - ☐ Performance monitoring enabled
 - ☐ Security monitoring enabled
 - ☐ **Testing**
 - ☐ Security tests written
 - ☐ Penetration testing completed
 - ☐ OWASP top 10 reviewed
 - ☐ Security audit completed
 - ☐ Incident response plan tested
-

Post-Launch Security Tasks

- ☐ Monitor error rates and security alerts
 - ☐ Review logs for suspicious activity
 - ☐ Conduct monthly security reviews
 - ☐ Update dependencies regularly
 - ☐ Perform quarterly penetration testing
 - ☐ Review and update security policies
 - ☐ Train team on security best practices
 - ☐ Conduct annual security audit
-

Document Status: Ready for Implementation

Last Updated: January 6, 2026

Next Review: After production deployment